GUÍA DE ESTUDIO · DESDE CERO

Entender el código de Raíces del Fogón

HTML · CSS · JavaScript · React · PHP · MySQL

Preparada para **Valentina Combita Durango**

Técnico en Programación de Software — SENA · Ficha 3166425

Colegio Parroquial San Luis Gonzaga

Cada tema se explica primero en general y después
con el código real del proyecto, para poder sustentarlo ante los profesores.

Contenido

Parte 0 Cómo usar esta guía y plan de estudio

Parte 1 ¿Qué es una aplicación web? El panorama completo

Parte 2 HTML: la estructura

Parte 3 CSS: el diseño

Parte 4 JavaScript: la lógica en el navegador

Parte 5 React: componentes, estado y rutas (¿es un framework?)

Parte 6 Cómo hablan el frontend y el backend: HTTP, API REST y JSON

Parte 7 PHP: el servidor

Parte 8 Bases de datos y MySQL: modelo relacional, DDL y DML

Parte 9 Validaciones y seguridad

Parte 10 La matriz CRUD del proyecto

Parte 11 Recorrido completo: la vida de un pedido

Parte 12 Mapa de archivos del proyecto

Parte 13 Preguntas probables de los profesores (con respuestas)

Parte 14 Glosario y autoevaluación

LO MÁS IMPORTANTE EN UNA FRASE

Raíces del Fogón es una **aplicación web** con tres capas: la **interfaz** (lo que se ve, hecha con React, que produce HTML, CSS y JavaScript), la **API** (un programa en PHP que recibe peticiones y aplica las reglas) y la **base de datos** (MySQL, donde se guardan platos, usuarios y pedidos).

PARTE 0

Cómo usar esta guía

Esta guía está pensada para alguien que empieza **desde cero**. No hace falta memorizar todo el código: lo que los profesores evalúan es que **entiendas qué hace cada parte, por qué está ahí y cómo se conectan**. Por eso cada tema tiene la misma estructura:

1. **El concepto** explicado con palabras sencillas.
2. **Un ejemplo pequeño** para entenderlo.
3. **Dónde está en el proyecto**, con el código real y el archivo donde vive.
4. **Cómo decirlo en la sustentación** (cuadros verdes).

Tipos de cuadros

> **CLAVE**
> Idea que tienes que dominar sí o sí.

> **ANALOGÍA**
> Comparación con algo cotidiano (casi siempre con el restaurante).

> **DILO ASÍ EN LA SUSTENTACIÓN**
> Una frase lista para responder a un profesor.

> **OJO**
> Error común o pregunta trampa.

Plan de estudio sugerido (7 días, 1–2 horas diarias)

Día	Tema	Práctica con el proyecto
1	Partes 1 y 2: aplicación web y HTML	Abrir la app, usar F12 → pestaña *Elements* y ver el HTML que genera React.
2	Parte 3: CSS	Abrir `src/index.css`, cambiar un color, ver el cambio, deshacerlo. Achicar la ventana para ver el diseño móvil.
3	Parte 4: JavaScript	Leer las funciones del carrito en `src/App.jsx` y explicarlas en voz alta.
4	Parte 5: React	Leer `Tarjeta.jsx`, `Filtro.jsx` y `Menu.jsx`. Identificar props, estado y eventos.
5	Partes 6 y 7: API y PHP	F12 → pestaña *Network*: agregar un plato al pedido y ver la petición y la respuesta JSON. Leer `api/rutas/categorias.php`.
6	Parte 8: MySQL	Ejecutar `database/consultas.sql` en phpMyAdmin o Workbench, una consulta a la vez.
7	Partes 9 a 13: seguridad, CRUD, recorrido y preguntas	Explicar en voz alta "la vida de un pedido" (Parte 11) y responder las preguntas de la Parte 13 sin mirar.

Cómo encender el proyecto para practicar

Se necesitan **Node.js** (para React), **PHP** y **MySQL** (por ejemplo, con XAMPP o Laragon). En una terminal, dentro de la carpeta del proyecto:

```
# 1. Descargar las librerías de JavaScript (solo la primera vez)
npm install
# 2. Copiar .env.example como .env y escribir la clave de MySQL y la del administrador
# 3. Crear la base de datos con datos de prueba
npm run db:setup:demo
# 4. Encender la API (déjala abierta)
npm run api
# 5. En OTRA terminal, encender la interfaz
npm run dev        # y abrir http://localhost:5173
```

> **DILO ASÍ**
> "El proyecto tiene dos programas que corren al tiempo: Vite sirve la interfaz de React en el puerto 5173 y PHP sirve la API en el puerto 3001. Vite reenvía las peticiones que empiezan por `/api` al servidor PHP."

PARTE 1

¿Qué es una aplicación web?

Una **página web** es un documento que el navegador (Chrome, Edge, Firefox) descarga y muestra. Una **aplicación web** es una página que además **hace cosas**: guarda datos, inicia sesión, registra pedidos. Para eso necesita varias piezas que trabajan juntas.

Cliente y servidor

- **Cliente:** el navegador del usuario. Pide cosas y muestra resultados.
- **Servidor:** un computador (o programa) que recibe esas peticiones, las procesa y responde.
- Se comunican por **Internet** usando un idioma llamado **HTTP**.

Las tres capas

Capa	Qué hace	Tecnología en Raíces del Fogón	Carpeta
Frontend (interfaz)	Lo que el usuario ve y toca: botones, formularios, menú.	HTML, CSS, JavaScript con **React**	`src/`
Backend (servidor, API)	Recibe peticiones, valida, aplica reglas de negocio y habla con la base de datos.	**PHP 8**	`api/`
Base de datos	Guarda la información de forma permanente y organizada.	**MySQL 8**	`database/`

> **ANALOGÍA DEL RESTAURANTE**
> El **frontend** es el **salón y la carta**: lo que el cliente ve. La **API** es el **mesero**: toma el pedido, revisa que tenga sentido y lo lleva a la cocina. El **backend** es la **cocina**, donde están las reglas ("no se sirve un plato agotado"). La **base de datos** es la **despensa y el cuaderno de pedidos**: donde todo queda guardado.



Figura 1. Arquitectura de Raíces del Fogón: el navegador pide, la API procesa y la base de datos guarda.

¿Qué es un "sistema de información"?

Es un conjunto de componentes que **recogen, guardan, procesan y entregan información** para apoyar una organización. En este caso: el restaurante registra sus platos, los clientes hacen pedidos, el administrador cambia su estado y consulta reportes de ventas.

Usuarios del sistema (roles)

Rol	Qué puede hacer
Visitante (sin cuenta)	Ver el menú, buscar, filtrar por categoría, ver "Nosotros", registrarse.
Cliente	Todo lo anterior + armar una orden, confirmar pedidos, ver "Mis pedidos" y cancelar los pendientes.
Administrador	Gestionar platos y categorías (crear, ver, editar, eliminar), subir fotos, cambiar el estado de los pedidos y ver reportes.

> **DILO ASÍ**
> "El sistema tiene arquitectura cliente-servidor en tres capas: una interfaz en React que corre en el navegador, una API REST en PHP que aplica las reglas y una base de datos relacional en MySQL. Se comunican con HTTP y JSON."

PARTE 2

HTML: la estructura

HTML (HyperText Markup Language) es el lenguaje que describe **qué hay** en una página: títulos, párrafos, imágenes, botones, formularios. **No es un lenguaje de programación** (no tiene lógica ni condiciones); es un lenguaje de **marcado**.

Etiquetas y atributos

HTML se escribe con **etiquetas**. Casi todas abren y cierran. Los **atributos** dan información extra a la etiqueta.

```
<a href="/registro">Regístrese aquí</a>
 |   └── atributo     └── contenido    └── etiqueta de cierre
 └── etiqueta de apertura

<img src="bandeja.jpg" alt="Bandeja paisa">   ← etiqueta vacía: no tiene cierre
```

Estructura básica de un documento

Este es el **único archivo .html** del proyecto. React se encarga de llenar el `div` con id `root`:

index.html

```
<!doctype html>                          ← le dice al navegador que es HTML5
<html lang="es">                         ← idioma español
  <head>                                 ← información que NO se ve
    <meta charset="UTF-8" />             ← permite tildes y la ñ
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />  ← clave para
celulares
    <title>Raíces del Fogón</title>  ← texto de la pestaña
  </head>
  <body>                                 ← lo que SÍ se ve
    <div id="root"></div>                ← aquí React "pinta" toda la aplicación
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

Etiquetas semánticas de HTML5

HTML5 trajo etiquetas que dicen **qué significa** cada zona, no solo cómo se ve. Ayudan a los buscadores y a las personas que usan lectores de pantalla.

Etiqueta	Significado	Dónde se usa en el proyecto
`<header>`	Encabezado	La barra superior (`Navbar.jsx`)
`<nav>`	Navegación	Los enlaces Menú, Nosotros, Mis pedidos…
`<main>`	Contenido principal	`App.jsx`
`<section>`	Sección temática	Misión, visión, valores (`Nosotros.jsx`)
`<article>`	Contenido independiente	Detalle del plato (`PlatoDetalle.jsx`)
`<footer>`	Pie de página	"Raíces del Fogón · Sabores de nuestra tierra"
`<ul>`, `<ol>`, `<li>`	Listas	El menú es una lista `<ul>` de tarjetas `<li>`
`<table>`	Tabla de datos	Listados del administrador y reportes

Formularios y controles HTML5

Los formularios permiten que el usuario escriba datos. HTML5 trae **tipos de campo** y **validaciones** incorporadas.

Control / atributo	Para qué sirve	Ejemplo en el proyecto
`<form>`	Agrupa los campos y se "envía"	Login, registro, plato, categoría
`<label>`	Texto que describe un campo (al hacer clic, enfoca el campo)	Todos los campos (componente `Campo.jsx`)
`type="email"`	Campo de correo	Login y registro
`type="password"`	Oculta lo que se escribe	Contraseña
`type="number"`	Solo números	Precio del plato
`type="search"`	Caja de búsqueda	Buscador del menú
`type="date"`	Selector de fecha	Filtro de pedidos por fecha
`type="file"`	Elegir un archivo	Foto del plato
`type="checkbox"`	Casilla sí/no	"Disponible" en el plato
`<select>`	Lista desplegable	Categoría del plato, estado del pedido
`<textarea>`	Texto largo	Descripción, observaciones del pedido
`required`	Campo obligatorio	Nombre, correo, contraseña…
`maxLength`	Largo máximo	Buscador (100), observaciones…
`alt` (en imágenes)	Texto alternativo si la imagen no carga o para lectores de pantalla	Foto de cada plato

OJO: PREGUNTA PROBABLE

"¿Dónde está el HTML si casi todo son archivos .jsx?" — En React, el HTML se escribe **dentro de JavaScript** con una sintaxis llamada **JSX**. Vite la convierte en instrucciones que crean las etiquetas HTML reales en el navegador. Se puede comprobar con F12 → *Elements*: ahí aparecen los `<header>`, `<form>`, `<input>`, etc.

Diferencias de JSX frente a HTML normal

HTML	JSX (React)	Por qué
`class="boton"`	`className="boton"`	`class` es palabra reservada de JavaScript
`for="correo"`	`htmlFor="correo"`	`for` también es palabra reservada
`onclick="..."`	`onClick={funcion}`	Los eventos van en camelCase y reciben una función
Texto fijo	`{plato.nombre}`	Las llaves `{ }` insertan valores de JavaScript

DILO ASÍ

"Uso HTML5 semántico: header, nav, main, section y footer. En los formularios uso label para cada campo, tipos como email, password, number, date y file, y atributos como required y maxLength. Como trabajo con React, el HTML está escrito en JSX, que se transforma en HTML real en el navegador."

PARTE 3

CSS: el diseño

CSS (Cascading Style Sheets, hojas de estilo en cascada) dice **cómo se ve** cada cosa: colores, tamaños, márgenes, posiciones, animaciones. HTML es el esqueleto; CSS es la ropa.

Anatomía de una regla CSS

```
.filtros button {            ← SELECTOR: a quién se aplica
  padding: 10px 25px;        ← PROPIEDAD: valor;
  border-radius: 25px;       ← esquinas redondeadas
  border: 2px solid #7b3f00; ← borde café
  cursor: pointer;           ← manito al pasar el mouse
  transition: all 0.3s;      ← cambios suaves en 0,3 segundos
}
```

Selectores principales

Selector	Se aplica a	Ejemplo del proyecto
`*`	Todos los elementos	Reinicio de márgenes
`body`	Una etiqueta	Fondo crema y letra Georgia
`.tarjeta`	Elementos con esa **clase**	Cada plato del menú
`#root`	El elemento con ese **id** (único)	Contenedor de React
`.filtros button`	Botones **dentro** de .filtros	Botones de categoría
`button:hover`	Cuando el mouse está encima	Botón se pone café
`:focus-visible`	Elemento enfocado con el teclado	Contorno dorado (accesibilidad)

El modelo de caja

Todo elemento en una página es una **caja** con cuatro capas, de adentro hacia afuera:

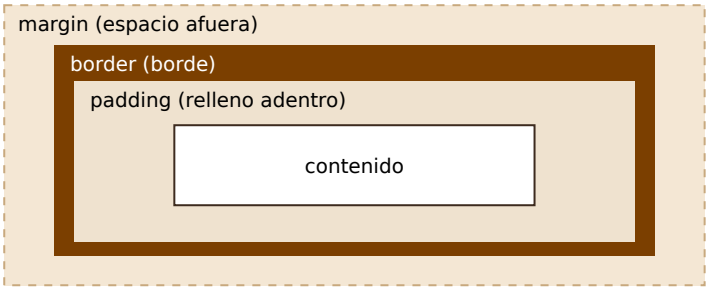


Figura 2. Modelo de caja: contenido, padding, border y margin.

El proyecto empieza con esta regla para que los tamaños sean predecibles:

src/index.css

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;    ← el ancho INCLUYE padding y borde
}
body {
  background-color: #fdf6ec;  ← crema
  font-family: 'Georgia', serif;
  color: #3d2b1f;             ← café oscuro
}
```

Colores de la marca

Color	Código	Uso
Café	`#7b3f00`	Títulos, botones, bordes
Café oscuro	`#3d2b1f`	Texto
Dorado arena	`#c8a97e`	Líneas, detalles, foco
Crema	`#fdf6ec`	Fondo

Los colores se escriben en **hexadecimal**: `#RRGGBB` (rojo, verde, azul), cada par de 00 a FF. La letra serif (Georgia) da un aire tradicional, acorde con la cocina colombiana de siempre.

Flexbox: acomodar en una fila o columna

`display: flex` convierte un contenedor en una "fila flexible": sus hijos se ponen uno al lado del otro y se pueden alinear y separar fácilmente.

```
.filtros {
  display: flex;              ← hijos en fila
  justify-content: center;    ← centrados horizontalmente
  gap: 15px;                  ← 15px de espacio entre cada botón
}
.menu {
  display: flex;
  flex-wrap: wrap;            ← si no caben, bajan a la siguiente línea
  gap: 25px;
  list-style: none;           ← quita las viñetas de la lista
}
```

Grid: acomodar en cuadrícula

`display: grid` organiza en filas **y** columnas. El proyecto lo usa para las tarjetas de valores y servicios:

```
grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
/* "Haz tantas columnas como quepan, cada una de mínimo 200px
    y que se repartan el espacio sobrante por igual (1fr)". */
```

Diseño responsivo (celular, tablet, computador)

Las **media queries** aplican estilos solo cuando la pantalla cumple una condición. Así la misma página se adapta al celular.

src/index.css

```
@media (max-width: 768px) {          ← solo en pantallas de 768px o menos
  .menu-movil { display: block; }    ← aparece el botón "Menú"
  .navegacion { display: none; flex-direction: column; }  ← se esconden los enlaces
  .navegacion.abierta { display: flex; }  ← y se muestran al tocar "Menú"
  .tabla thead { display: none; }    ← las tablas se vuelven "tarjetas"
}
```

Otras propiedades que aparecen

- `position: sticky`: el carrito se queda pegado al bajar la página.
- `transition`: los cambios de color o tamaño se ven suaves.
- `transform`: la tarjeta "sube" un poco al pasar el mouse.
- `border-radius`: esquinas redondeadas; `box-shadow`: sombras.
- Unidades: `px` (píxeles fijos), `rem` (relativo al tamaño de letra base), `%`, `fr` (fracción en grid).

OJO

En la carpeta `src/` hay archivos que vinieron con la plantilla de Vite y **no se usan**: `App.css`, `assets/react.svg`, `assets/vite.svg` y `assets/hero.png`. Los estilos reales están en `src/index.css`. Si un profesor pregunta, esa es la respuesta (o mejor, borrarlos antes de sustentar).

DILO ASÍ

"Todos los estilos están en `src/index.css`. Uso selectores de clase, Flexbox para filas como los filtros y el menú, Grid para las rejillas de valores y servicios, y media queries para que el sitio se adapte al celular: en pantallas pequeñas aparece un botón 'Menú' y las tablas se convierten en tarjetas. Los colores café, crema y dorado representan la identidad de la marca."

PARTE 4

JavaScript: la lógica en el navegador

JavaScript (JS) es un **lenguaje de programación**: tiene variables, condiciones, ciclos y funciones. En el navegador es lo que hace que la página **reaccione**: sumar al carrito, validar un formulario, pedir datos al servidor.

> **OJO**
> JavaScript **no es** Java. Son lenguajes totalmente distintos; solo comparten parte del nombre.

Variables

```
const nombre = "Bandeja paisa"   // const: su valor no se reasigna
let cantidad = 1                 // let: su valor puede cambiar
cantidad = cantidad + 1          // ahora vale 2
```

Tipos de datos

Tipo	Ejemplo
Texto (string)	`"Arroz con leche"`
Número (number)	`28000`, `9.5`
Booleano (boolean)	`true` / `false`
Vacío	`null`, `undefined`
Objeto (grupo de datos con nombre)	`{ id: 5, nombre: "Bandeja paisa", precio: 28000 }`
Arreglo (lista ordenada)	`[plato1, plato2, plato3]`

Un plato que llega de la API es un **objeto**; el carrito es un **arreglo de objetos**. Se accede a un dato con punto: `plato.precio`.

Funciones

Una función es un bloque de código con nombre que se puede reutilizar. Hay dos formas de escribirlas:

```
function sumar(a, b) {        // forma clásica
  return a + b
}
const sumar2 = (a, b) => a + b  // "función flecha" (arrow function)
```

Condiciones

```
if (existente) {
  // si ya está en el carrito
} else {
  // si no está
}
// Operador ternario: condición ? siVerdadero : siFalso
agregado ? "Agregado" : "Agregar a la orden"
```

Métodos de arreglos (muy usados en el proyecto)

Método	Qué hace	Ejemplo
`find`	Devuelve el **primer** elemento que cumple la condición	`carrito.find(item => item.id === plato.id)`
`filter`	Devuelve un arreglo **solo con** los que cumplen	Quitar un plato del carrito
`map`	Transforma **cada** elemento en otro	Convertir cada plato en una tarjeta
`reduce`	Acumula todos en **un solo valor**	Calcular el total del pedido

El carrito explicado línea por línea

src/App.jsx

```
function agregarAlCarrito(plato) {
  // 1. ¿Este plato ya está en el carrito?
  const existente = carrito.find((item) => item.id === plato.id)

  if (existente) {
    if (existente.cantidad >= 50) return      // límite de 50 unidades
    // 2. Si ya está, se crea un carrito nuevo donde ESE plato tiene cantidad + 1
    setcarrito(carrito.map((item) =>
      item.id === plato.id
        ? { ...item, cantidad: item.cantidad + 1 }   // copia del plato con +1
        : item))                                      // los demás quedan igual
  } else {
    // 3. Si no está, se agrega al final con cantidad 1
    setcarrito([...carrito, { ...plato, cantidad: 1 }])
  }
}

// Total de unidades: se empieza en 0 y se suma la cantidad de cada item
const cantidadOrden = carrito.reduce((suma, item) => suma + item.cantidad, 0)
```

> **LOS TRES PUNTOS (...) — "SPREAD"**
> `...item` copia todas las propiedades de un objeto; `...carrito` copia todos los elementos de un arreglo. En React **no se modifica** el carrito directamente: se crea uno nuevo con los cambios. Así React detecta que algo cambió y vuelve a dibujar la pantalla.

Desestructuración

```
const { name, value } = evento.target
// es lo mismo que:  const name = evento.target.name;  const value = evento.target.value
```

Plantillas de texto

```
`/mis-pedidos/${pedido.id}`   // con comillas invertidas y ${ } se mete un valor en el texto
```

Operadores útiles

- `===` compara valor **y** tipo (siempre se prefiere a `==`).
- `&&`: "y"; en JSX se usa para mostrar algo solo si se cumple una condición.

- `?.` (encadenamiento opcional): `usuario?.rol` no da error si `usuario` es `null`.
- `||` y `??`: dar un valor por defecto.

Asincronía: async / await y fetch

Pedir datos a un servidor **tarda**. JavaScript no se queda congelado esperando: usa **promesas**. Con `async` / `await` se escribe como si fuera paso a paso:

```
async function cargar() {
  try {
    const respuesta = await fetch('/api/platos')   // espera la respuesta del servidor
    const platos = await respuesta.json()          // convierte el JSON en objetos de JS
  } catch (error) {
    // si no hay conexión, se cae aquí y se muestra un mensaje
  }
}
```

`fetch` es la función del navegador para hacer peticiones HTTP. `try/catch` atrapa errores para que la app no se rompa.

Módulos: import y export

El código se divide en archivos. Un archivo **exporta** lo que ofrece y otro lo **importa**:

```
// src/utils/formato.js
export function precio(valor) { ... }

// src/Tarjeta.jsx
import { precio } from "./utils/formato"
```

Expresiones regulares (regex)

Son **patrones** para revisar el formato de un texto. Se usan en las validaciones:

src/utils/validaciones.js

```
const REGEX_TELEFONO = /^\+?\d{7,15}$/
// ^      inicio del texto
// \+?    un "+" opcional
// \d{7,15}  entre 7 y 15 dígitos
// $      fin del texto   →  "3001234567" ✔   "300-abc" ✘
```

> **DILO ASÍ**
> "JavaScript maneja la lógica de la interfaz. Por ejemplo, el carrito es un arreglo de objetos: uso `find` para saber si el plato ya está, `map` para aumentar la cantidad, `filter` para quitarlo y `reduce` para calcular el total. Para hablar con el servidor uso `fetch` con `async/await`."

PARTE 5

React: componentes, estado y rutas

¿Qué es React? ¿Es un framework?

React es una herramienta de JavaScript, creada por Meta (Facebook), para construir **interfaces de usuario** dividiéndolas en piezas reutilizables llamadas **componentes**.

> **PREGUNTA TRAMPA: ¿REACT ES UN FRAMEWORK O UNA LIBRERÍA?**
> Técnicamente, React se define a sí mismo como una **librería (biblioteca)**: solo se encarga de la interfaz. Un **framework** da la estructura completa de la aplicación y "decide" cómo se organiza todo. En este proyecto React se combina con **React Router** (navegación entre páginas) y **Vite** (herramienta de desarrollo), y juntos funcionan como un **ecosistema de tipo framework**. Respuesta segura: *"React es una librería de JavaScript para interfaces; en el lenguaje común se le llama framework porque, junto con React Router y Vite, cumple ese papel."*

	Librería	Framework
Quién manda	Tu código llama a la librería cuando la necesita	El framework llama a tu código (te impone la estructura)
Ejemplos	React, jQuery	Angular, Laravel, Next.js

¿Por qué usar React y no solo HTML + JS?

- **Componentes reutilizables:** `Tarjeta` se escribe una vez y se usa para los 9 platos.
- **Interfaz automática:** cuando cambian los datos (el estado), React actualiza la pantalla solo.
- **SPA (Single Page Application):** la página se carga una vez y luego se cambia de "pantalla" sin recargar, así se siente rápida.

Herramientas alrededor de React

Herramienta	Qué es
Node.js	Permite ejecutar JavaScript fuera del navegador. Se necesita para las herramientas de desarrollo.
npm	Gestor de paquetes: descarga librerías (`npm install`) y ejecuta los scripts.
`package.json`	"Ficha técnica" del proyecto: nombre, librerías que usa y comandos (`dev`, `api`, `build`...).
`node_modules/`	Carpeta donde npm guarda las librerías descargadas. No se sube a GitHub.
Vite	Servidor de desarrollo (recarga al guardar) y "compilador": `npm run build` convierte el JSX en HTML/CSS/JS listos en la carpeta `dist/`.
React Router	Librería para tener varias "páginas" (rutas) dentro de la SPA.
ESLint	Revisa el código en busca de errores (`npm run lint`).

1. Componentes

Un componente es **una función que devuelve JSX** (lo que se va a mostrar). Su nombre empieza con mayúscula.

src/Tarjeta.jsx

```
function Tarjeta({ plato, agregarAlCarrito }) {          ← recibe "props"
  const [agregado, setAgregado] = useState(false)        ← estado propio

  function anima() {
    agregarAlCarrito(plato)                              ← avisa al padre
    setAgregado(true)                                    ← el botón dice "Agregado"
    setTimeout(() => { setAgregado(false) }, 1000)       ← y a 1 segundo vuelve
  }

  return (
    <li className="tarjeta">
      {plato.imagen && <img src={plato.imagen} alt={plato.nombre} loading="lazy" />}
      <h3>{plato.nombre}</h3>
      <p>{plato.descripcion}</p>
      <p className="precio">{precio(plato.precio)}</p>
      <button onClick={anima}>{agregado ? "Agregado" : "Agregar a la orden"}</button>
    </li>
  )
}
export default Tarjeta
```

2. Props: datos que el padre le pasa al hijo

Las **props** son como los **parámetros** de una función. El componente padre se los pasa al hijo como atributos:

```
// En Menu.jsx (el padre) se crea una Tarjeta por cada plato:
{platos.map((plato) => (
  <Tarjeta key={plato.id} plato={plato} agregarAlCarrito={agregarAlCarrito} />
))}
```

`key` es un identificador único que React necesita en las listas para saber qué elemento es cuál.

> **ANALOGÍA**
> Las props son la **comanda** que el mesero le entrega al cocinero: el cocinero (componente hijo) no decide qué plato hacer, solo lo prepara con los datos que recibe.

3. Estado (useState): la memoria del componente

El **estado** son datos que **cambian** con el tiempo y que, al cambiar, hacen que la pantalla se actualice.

```
const [carrito, setcarrito] = useState([])
//     |         |                 └── valor inicial: arreglo vacío
//     |         └── función para cambiarlo
//     └── valor actual
```

> **PROPS VS. ESTADO**
> **Props:** vienen de afuera (del padre) y el hijo no las cambia. **Estado:** es interno del componente y él mismo lo cambia con su función `set…`.

4. Eventos

```
<button onClick={anima}>             ← al hacer clic, ejecuta anima
<input onChange={(e) => setBuscar(e.target.value)} />    ← al escribir, guarda el texto
<form onSubmit={enviar}>              ← al enviar el formulario
```

5. Renderizado condicional

```
{cargando && <p>Cargando menú…</p>}                        ← solo si está cargando
{carrito.length > 0 && <Carrito carrito={carrito} ... />}  ← el carrito aparece si tiene algo
```

6. useEffect: hacer algo cuando el componente aparece o cambia algo

`useEffect` sirve para **efectos secundarios**, sobre todo **pedir datos al servidor**.

`src/pages/Menu.jsx`

```
// Se ejecuta UNA vez al abrir el menú (arreglo vacío [] al final)
useEffect(() => {
  api.categorias().then(setCategorias).catch(() => setCategorias([]))
}, [])

// Se ejecuta cada vez que cambia la categoría o el texto de búsqueda
useEffect(() => {
  let vigente = true
  const espera = setTimeout(() => {                  ← espera 250 ms ("debounce")
    api.platos({ categoria_id: ..., buscar })
      .then((datos) => { if (vigente) setPlatos(datos) })
  }, 250)
  return () => { vigente = false; clearTimeout(espera) }  ← limpieza
}, [categoria, buscar])                              ← "dependencias"
```

El **debounce** evita hacer una petición por cada letra que se escribe: espera a que la persona deje de escribir un momento. La bandera `vigente` descarta respuestas viejas que lleguen tarde.

7. Context: datos compartidos por toda la app (la sesión)

Pasar props de padre a hijo a nieto es incómodo. **Context** crea un "almacén" que cualquier componente puede leer. El proyecto lo usa para saber **quién inició sesión**.

src/context/SesionProvider.jsx (resumen)

```
function SesionProvider({ children }) {
  const [usuario, setUsuario] = useState(null)
  useEffect(() => {
    api.sesion().then((datos) => setUsuario(datos.usuario))  ← ¿hay sesión abierta?
  }, [])
  const valor = { usuario, esAdmin: usuario?.rol === 'admin', iniciarSesion, cerrarSesion, ... }
  return <SesionContext.Provider value={valor}>{children}</SesionContext.Provider>
}

// En cualquier componente:
const { usuario, esAdmin } = useSesion()
```

8. React Router: las "páginas"

src/App.jsx (fragmento)

```
<Routes>
  <Route path="/"            element={<Menu ... />} />
  <Route path="/nosotros"    element={<Nosotros />} />
  <Route path="/login"       element={<Login />} />
  <Route path="/mis-pedidos" element={<RutaProtegida><MisPedidos /></RutaProtegida>} />
  <Route path="/admin/platos" element={<RutaProtegida soloAdmin><AdminPlatos /></RutaProtegida>}
/>
  <Route path="/admin/platos/:id/editar" element={...} />   ← :id es un parámetro
  <Route path="*" element={<p>La página que busca no existe.</p>} />  ← página 404
</Routes>
```

`RutaProtegida` es un componente "guardia": si no hay sesión, manda al login; si la ruta es `soloAdmin` y el usuario es cliente, muestra "No tiene permisos".

Cómo arranca todo

src/main.jsx

```
createRoot(document.getElementById('root')).render(   ← busca el div#root del index.html
  <StrictMode>           ← modo estricto: avisa de errores en desarrollo
    <BrowserRouter>      ← activa las rutas
      <SesionProvider>   ← activa la sesión para toda la app
        <App />          ← la aplicación
      </SesionProvider>
    </BrowserRouter>
  </StrictMode>
)
```

Árbol de componentes

```
main.jsx
└── App.jsx  (guarda el carrito y define las rutas)
    ├── Navbar          (barra superior, muestra cantidad de la orden)
    ├── Menu            ruta "/"
    │   ├── Filtro      (botones de categoría)
    │   ├── Tarjeta ×N  (un plato)
    │   └── Carrito     (la orden, observaciones, confirmar)
    ├── Nosotros, Registro, Login
    ├── MisPedidos, PedidoDetalle         (cliente)
    └── AdminPlatos, PlatoFormulario, PlatoDetalle,
        AdminCategorias, AdminPedidos, Reportes   (administrador)
Componentes pequeños reutilizables: Campo, Aviso, EstadoPedido, RutaProtegida
```

DILO ASÍ

"La interfaz está hecha con React, una librería de JavaScript basada en componentes. Cada parte de la pantalla es un componente, por ejemplo `Tarjeta` muestra un plato. Los componentes reciben datos por props y guardan su propia información con el hook `useState`. Con `useEffect` pido los datos a la API, con Context comparto la sesión del usuario y con React Router manejo las páginas, incluyendo rutas protegidas para el administrador."

PARTE 6

Cómo hablan el frontend y el backend

HTTP: el idioma de la web

Cada vez que la interfaz necesita algo del servidor envía una **petición HTTP** y recibe una **respuesta**. Una petición tiene:

- **Método** (qué quiero hacer) y **ruta** (sobre qué): `GET /api/platos`
- **Cabeceras** (información extra, como la cookie de sesión).
- **Cuerpo** opcional (los datos que envío, en JSON).

Métodos HTTP y su relación con el CRUD

Método	Significado	CRUD	Ejemplo
`GET`	Consultar	Read	`GET /api/platos`
`POST`	Crear	Create	`POST /api/pedidos`
`PUT`	Reemplazar / editar todo	Update	`PUT /api/platos/5`
`PATCH`	Editar una parte	Update	`PATCH /api/pedidos/3/estado`
`DELETE`	Eliminar	Delete	`DELETE /api/categorias/4`

Códigos de estado (la respuesta trae un número)

Código	Significado	Cuándo lo usa el proyecto
200	OK	Consulta o edición exitosa
201	Creado	Se registró un usuario, plato o pedido
400	Petición incorrecta	Datos del formulario inválidos
401	No autenticado	No ha iniciado sesión / contraseña incorrecta
403	Prohibido	Un cliente intenta hacer algo de administrador
404	No encontrado	El plato o pedido no existe
409	Conflicto	Correo repetido; borrar categoría con platos
429	Demasiadas peticiones	Más de 5 intentos de login fallidos
500	Error del servidor	Algo inesperado falló

JSON: el formato de los datos

JSON (JavaScript Object Notation) es texto con la misma forma que los objetos de JavaScript. Es el formato en que viajan los datos entre React y PHP:

```
{ "id": 5, "nombre": "Bandeja paisa", "precio": 28000, "categoria": "Plato fuerte",
"disponible": true }
```

¿Qué es una API REST?

Una **API** (Interfaz de Programación de Aplicaciones) es un conjunto de "puertas" que un programa ofrece para que otro lo use. Es **REST** porque organiza esas puertas por **recursos** (platos, pedidos, categorías) y usa los **métodos HTTP** para indicar la acción.

Endpoints de la API de Raíces del Fogón

Método y ruta	Quién	Qué hace
POST /api/auth/registro	Público	Crea una cuenta de cliente
POST /api/auth/login	Público	Inicia sesión
POST /api/auth/logout	Público	Cierra sesión
GET /api/auth/sesion	Público	¿Quién está conectado?
GET /api/categorias · GET /api/categorias/{id}	Público	Lista / una categoría
POST · PUT · DELETE /api/categorias	Admin	Crear, editar, eliminar categoría
GET /api/platos · GET /api/platos/{id}	Público*	Lista con filtros / un plato
POST · PUT · DELETE /api/platos	Admin	Crear, editar, eliminar plato
POST · DELETE /api/platos/{id}/imagen	Admin	Subir / quitar foto
POST /api/pedidos	Cliente	Crear pedido
GET /api/pedidos · GET /api/pedidos/{id}	Cliente / Admin	Cliente ve los suyos; admin ve todos
PATCH /api/pedidos/{id}/estado	Admin	Cambiar estado
PATCH /api/pedidos/{id}/cancelar	Cliente	Cancelar un pedido pendiente
GET /api/reportes/resumen	Admin	Datos de la página de reportes

* Los visitantes solo ven los platos disponibles; el administrador ve todos.

La capa de servicios en React

Todas las peticiones salen de **un solo archivo**. Así, si algo cambia, se cambia en un solo lugar:

`src/services/api.js (resumen)`

```
async function pedir(metodo, ruta, cuerpo) {
  const respuesta = await fetch(`/api${ruta}`, {
    method: metodo,
    credentials: 'same-origin',                  ← envía la cookie de sesión
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(cuerpo),                ← objeto de JS → texto JSON
  })
  const datos = await respuesta.json()
  if (!respuesta.ok) throw new ErrorApi(datos.error, respuesta.status, datos.campos)
  return datos
}

export const api = {
  platos: (filtros) => pedir('GET', `/platos${consulta(filtros)}`),
  crearPedido: (datos) => pedir('POST', '/pedidos', datos),
  cambiarEstado: (id, estado) => pedir('PATCH', `/pedidos/${id}/estado`, { estado }),
  ...
}
```

Sesión y cookies

HTTP "no tiene memoria": cada petición es independiente. Para recordar quién inició sesión, el servidor envía una **cookie** (un pequeño dato que el navegador guarda y reenvía en cada petición). El servidor la relaciona con una **sesión** donde guarda el id y el rol del usuario.

El proxy de Vite

vite.config.js

```
const api = {
  '/api': 'http://127.0.0.1:3001',       ← lo que empiece por /api se reenvía a PHP
  '/uploads': 'http://127.0.0.1:3001',   ← y las fotos también
}
```

Gracias a esto, para el navegador todo viene de la misma dirección y la cookie funciona sin configuraciones extra (CORS).

> **DILO ASÍ**
> "El frontend y el backend se comunican con una API REST. React hace peticiones HTTP con `fetch` desde un solo archivo, `services/api.js`; el servidor responde en JSON con un código de estado, por ejemplo 201 cuando se crea un pedido o 403 si un cliente intenta algo de administrador. La sesión se mantiene con una cookie segura."

PARTE 7

PHP: el servidor

PHP es un lenguaje de programación que se ejecuta **en el servidor**, no en el navegador. El usuario nunca ve el código PHP, solo el resultado (en este proyecto, JSON). Aquí PHP cumple el papel de **API**: recibe las peticiones, valida, consulta MySQL y responde.

Sintaxis básica

```
<?php                                   // todo archivo PHP empieza así
$nombre = "Bandeja paisa";              // las variables empiezan con $
$precio = 28000;
$plato = ['nombre' => $nombre, 'precio' => $precio];   // arreglo asociativo (como un objeto)
echo $plato['nombre'];                  // muestra "Bandeja paisa"

function sumar(int $a, int $b): int {   // función con tipos
    return $a + $b;
}
if ($precio > 20000) { ... } else { ... }
foreach ($platos as $p) { ... }         // recorrer una lista
const MINUTOS_BLOQUEO = 15;             // constante
```

Diferencias con JavaScript: las variables llevan `$`, las líneas terminan en `;`, se concatena texto con punto (`"Hola " . $nombre`) y los arreglos usan `=>`.

Organización del backend

Archivo	Responsabilidad
`api/index.php`	**Punto de entrada y enrutador**: toda petición entra aquí y se envía a la función correcta.
`api/lib/config.php`	Lee el archivo `.env` (usuario y clave de MySQL).
`api/lib/db.php`	Conexión a MySQL con PDO y funciones para consultar.
`api/lib/respuesta.php`	Responder en JSON, leer el cuerpo de la petición y manejar errores.
`api/lib/seguridad.php`	Sesiones y permisos (`requiereSesion`, `requiereAdmin`).
`api/lib/validaciones.php`	Reglas de validación de cada formulario.
`api/lib/archivos.php`	Recibir, revisar y guardar las fotos.
`api/rutas/*.php`	Los endpoints: `auth`, `categorias`, `platos`, `pedidos`, `reportes`.

El enrutador

Cada endpoint se registra con la función `ruta(método, patrón, acción)`. Después, `index.php` compara la petición con la lista y ejecuta la que coincide:

api/index.php (simplificado)

```php
function ruta(string $metodo, string $patron, callable $accion): void {
    global $RUTAS;
    // "/platos/{id}" se convierte en un patrón que acepta "/platos/5"
    $regex = '#^' . preg_replace('#\{\w+\}#', '([^/]+)', $patron) . '$#';
    $RUTAS[] = [$metodo, $regex, $accion];
}

foreach ($RUTAS as [$metodoRuta, $regex, $accion]) {
    if (preg_match($regex, $recurso, $parametros) && $metodoRuta === $metodo) {
        $accion(...$parametros);      // ejecuta la función de esa ruta
        exit;
    }
}
responder(['error' => 'Recurso no encontrado.'], 404);   // si ninguna coincide
```

Conexión a la base de datos con PDO

PDO (PHP Data Objects) es la forma moderna de conectarse a una base de datos desde PHP.

api/lib/db.php

```php
$pdo = new PDO('mysql:host=127.0.0.1;dbname=raices_del_fogon;charset=utf8mb4', $usuario, $clave,
[
    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,      // si algo falla, lanza un error
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC, // filas como arreglos con nombre
]);

function consultar(string $sql, array $parametros = []): array {
    $sentencia = conexion()->prepare($sql);   // 1. prepara la consulta con "?"
    $sentencia->execute($parametros);         // 2. envía los valores por separado
    return $sentencia->fetchAll();            // 3. devuelve todas las filas
}
```

> **CONSULTAS PREPARADAS = PROTECCIÓN CONTRA INYECCIÓN SQL**
> Los valores **nunca** se pegan dentro del texto SQL; van en los `?` y se envían aparte. Así, si alguien escribe `' OR 1=1 --` en el login, MySQL lo trata como un texto cualquiera y no como una orden.

Un CRUD completo en PHP: categorías

api/rutas/categorias.php

```
// READ: listar (público)
ruta('GET', '/categorias', function () {
    $categorias = consultar(SQL_CATEGORIAS . ' GROUP BY c.id ORDER BY c.id');
    responder(array_map('formatoCategoria', $categorias));
});

// CREATE: crear (solo administrador)
ruta('POST', '/categorias', function () {
    requiereAdmin();                                        ← 1. ¿es admin?
    $datos = validar('validarCategoria', cuerpo());         ← 2. ¿datos válidos?
    ejecutar('INSERT INTO categorias (nombre, descripcion) VALUES (?, ?)',
             [$datos['nombre'], $datos['descripcion']]);    ← 3. guardar
    responder(['id' => ultimoId()] + $datos, 201);          ← 4. responder 201
});

// UPDATE: editar
ruta('PUT', '/categorias/{id}', function (string $id) {
    requiereAdmin();
    $datos = validar('validarCategoria', cuerpo());
    ejecutar('UPDATE categorias SET nombre = ?, descripcion = ? WHERE id = ?',
             [$datos['nombre'], $datos['descripcion'], $id]);
    responder(['id' => $id] + $datos);
});

// DELETE: eliminar (solo si no tiene platos)
ruta('DELETE', '/categorias/{id}', function (string $id) {
    requiereAdmin();
    $uso = consultarUno('SELECT COUNT(*) AS total FROM platos WHERE categoria_id = ?', [$id]);
    if ($uso['total'] > 0) {
        throw new ErrorApi(409, "No se puede eliminar: la categoría tiene {$uso['total']}
plato(s).");
    }
    ejecutar('DELETE FROM categorias WHERE id = ?', [$id]);
    responder(['mensaje' => 'Categoría eliminada.']);
});
```

Todas las rutas siguen el mismo patrón: **1) permisos → 2) validar → 3) SQL → 4) responder JSON**.

Manejo de errores

Cuando algo sale mal se lanza una excepción `ErrorApi` con un código y un mensaje. La función `manejarError` la atrapa y la convierte en una respuesta JSON. También traduce errores de MySQL a mensajes entendibles: el error 1062 (valor duplicado) se vuelve "Ya existe un registro con ese valor" (409).

Transacciones: todo o nada

Al crear un pedido hay que guardar el pedido **y** todas sus líneas de detalle. Si falla a mitad, no puede quedar un pedido "a medias". Una **transacción** garantiza que se guarda todo o no se guarda nada:

api/rutas/pedidos.php (resumen)

```
$pedido = transaccion(function () use ($datos, $ids, $sesion) {
    // 1. Los precios se leen de la BD, NUNCA del navegador (nadie puede "cambiar" el precio)
    $platos = consultar("SELECT id, precio FROM platos WHERE disponible = 1 AND id IN (...)",
$ids);
    // 2. Calcular el total
    // 3. INSERT INTO pedidos (usuario_id, total, observaciones) ...
    // 4. INSERT INTO detalle_pedido (...) por cada plato
    return ['id' => $pedidoId, 'total' => $total, 'estado' => 'pendiente'];
});   // si algo falla → ROLLBACK (se deshace todo); si no → COMMIT (se confirma)
```

DILO ASÍ

"El backend es una API REST en PHP 8. Todas las peticiones entran por `api/index.php`, que funciona como enrutador. Cada ruta revisa permisos, valida los datos, ejecuta SQL con consultas preparadas de PDO para evitar inyección SQL y responde en JSON. El pedido se guarda dentro de una transacción para que nunca quede incompleto, y los precios se toman de la base de datos, no del navegador."

PARTE 8

Bases de datos y MySQL

Conceptos básicos

Concepto	Qué es	Ejemplo
Base de datos	Conjunto organizado de información	`raices_del_fogon`
SGBD	Programa que administra bases de datos	MySQL (o MariaDB en XAMPP)
Tabla	Conjunto de datos de un mismo tipo (una "entidad")	`platos`
Columna (campo, atributo)	Un dato que tienen todos los registros	`precio`
Fila (registro, tupla)	Un elemento concreto	La bandeja paisa
SQL	Lenguaje para hablar con la base de datos	`SELECT * FROM platos`

Una base de datos **relacional** guarda la información en tablas que se **relacionan** entre sí por medio de llaves.

Llaves y restricciones

Restricción	Significado	Ejemplo en el proyecto
PRIMARY KEY (PK)	Identifica cada fila de forma única; no se repite ni es nula	`id` en todas las tablas
FOREIGN KEY (FK)	Columna que apunta a la PK de otra tabla (crea la relación)	`platos.categoria_id → categorias.id`
UNIQUE	No se puede repetir	Correo del usuario, nombre del plato
NOT NULL	Obligatorio	Nombre, precio…
DEFAULT	Valor si no se envía nada	`estado` = 'pendiente', `disponible` = 1
CHECK	Condición que debe cumplirse	`precio > 0`; `cantidad BETWEEN 1 AND 50`
AUTO_INCREMENT	MySQL asigna el número solo (1, 2, 3…)	Todos los `id`
INDEX	Acelera las búsquedas por esa columna (como el índice de un libro)	`ix_pedidos_estado`

Tipos de datos usados

Tipo	Para qué	Ejemplo
`INT UNSIGNED`	Números enteros sin signo (no negativos)	ids
`SMALLINT`	Entero pequeño	cantidad
`VARCHAR(n)`	Texto de hasta n caracteres	nombre VARCHAR(100)
`DECIMAL(10,2)`	Números con decimales exactos (dinero)	precio, total
`TINYINT(1)`	Sí/No (0 o 1)	disponible
`ENUM(...)`	Solo uno de una lista de valores	rol: 'cliente' o 'admin'
`TIMESTAMP`	Fecha y hora	creado_en

Las 7 tablas del proyecto

Tabla	Guarda	Columnas principales
`categorias`	Tipos de plato	id, nombre, descripcion
`platos`	El menú	id, categoria_id (FK), nombre, descripcion, precio, imagen_url, archivo_id (FK), disponible
`archivos`	Fotos subidas	id, nombre_original, nombre_sistema, ruta, tipo_mime, tamano_bytes
`usuarios`	Clientes y administradores	id, nombre, correo, telefono, password_hash, rol
`pedidos`	Encabezado de cada pedido	id, usuario_id (FK), estado, total, observaciones, creado_en
`detalle_pedido`	Los platos de cada pedido	id, pedido_id (FK), plato_id (FK), cantidad, precio_unitario, subtotal
`intentos_login`	Intentos de inicio de sesión (seguridad)	id, correo, ip, creado_en

Modelo relacional

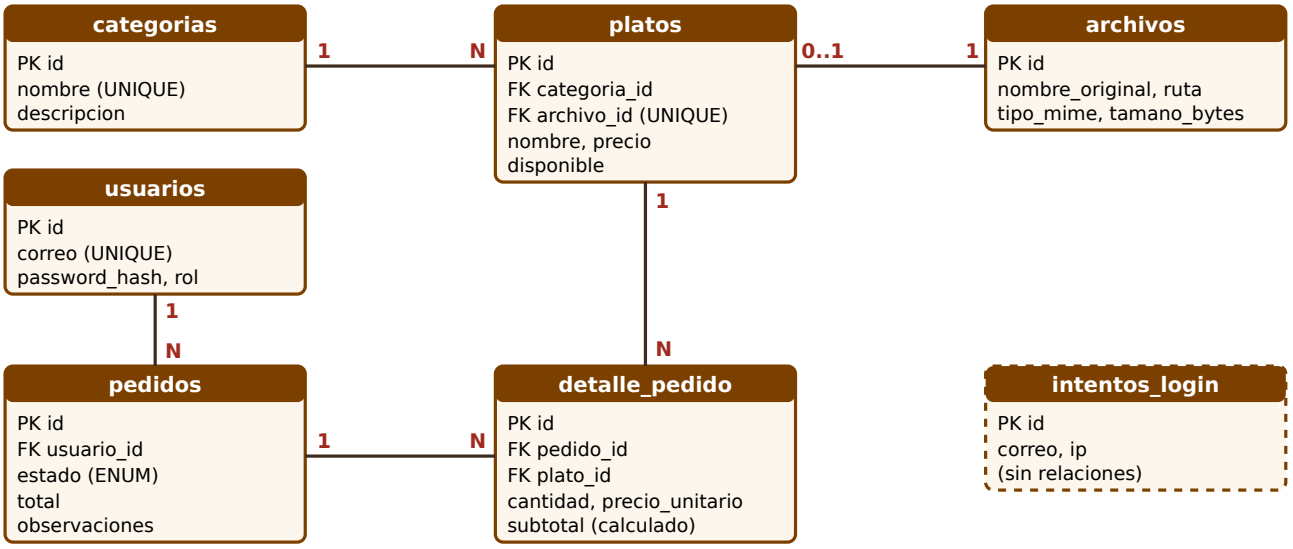


Figura 3. Modelo relacional. 1 = uno, N = muchos. La tabla intentos_login no se relaciona con las demás.

Cómo se leen las relaciones

- **categorias 1 — N platos:** una categoría tiene muchos platos; cada plato pertenece a una sola categoría.
- **usuarios 1 — N pedidos:** un usuario hace muchos pedidos; cada pedido es de un solo usuario.
- **pedidos 1 — N detalle_pedido** y **platos 1 — N detalle_pedido:** un pedido tiene muchos platos y un plato aparece en muchos pedidos. Esa relación **muchos a muchos** se resuelve con la **tabla intermedia** `detalle_pedido`.
- **platos 0..1 — 1 archivos:** un plato puede tener como máximo una foto subida (es opcional).

> **OJO**
> El diagrama de texto de la sección 9.1 de `DOCUMENTACION_TECNICA.md` dibuja `platos` unido a `pedidos`. Lo correcto es `platos` → `detalle_pedido`, como en la Figura 3. Conviene corregirlo antes de sustentar.

¿Por qué el detalle guarda `precio_unitario` si el plato ya tiene precio?

Porque los precios cambian. Si la bandeja paisa sube de $28.000 a $30.000, los pedidos viejos deben seguir mostrando lo que el cliente pagó. Por eso se copia el precio en el momento de la compra (**precio histórico**).

Reglas de las llaves foráneas

Regla	Qué pasa al borrar el registro "padre"	Dónde se usa
`ON DELETE RESTRICT`	**No deja borrar** si tiene registros relacionados	No se puede borrar una categoría con platos, ni un plato que ya se vendió
`ON DELETE CASCADE`	Borra también los hijos	Si se borra un pedido, se borra su detalle
`ON DELETE SET NULL`	Deja la FK vacía	Si se borra la foto, el plato queda sin foto
`ON UPDATE CASCADE`	Si cambia el id del padre, se actualiza en los hijos	Todas las FK

DDL: definir la estructura

DDL (Data Definition Language) son los comandos que **crean o cambian la estructura**: `CREATE`, `ALTER`, `DROP`. Están en `database/schema.sql`.

`database/schema.sql (tabla platos)`

```
CREATE DATABASE IF NOT EXISTS raices_del_fogon
  CHARACTER SET utf8mb4 COLLATE utf8mb4_spanish_ci;   -- utf8mb4: tildes, ñ y emojis

CREATE TABLE IF NOT EXISTS platos (
  id            INT UNSIGNED  NOT NULL AUTO_INCREMENT,
  categoria_id  INT UNSIGNED  NOT NULL,
  nombre        VARCHAR(100)  NOT NULL,
  descripcion   VARCHAR(500)  NOT NULL,
  precio        DECIMAL(10,2) NOT NULL,
  disponible    TINYINT(1)    NOT NULL DEFAULT 1,
  creado_en     TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT pk_platos PRIMARY KEY (id),
  CONSTRAINT uq_platos_nombre UNIQUE (nombre),
  CONSTRAINT ck_platos_precio CHECK (precio > 0),
  CONSTRAINT fk_platos_categoria FOREIGN KEY (categoria_id)
    REFERENCES categorias (id)
    ON UPDATE CASCADE
    ON DELETE RESTRICT
) ENGINE=InnoDB;     -- InnoDB: motor que soporta llaves foráneas y transacciones
```

Columna **calculada** en `detalle_pedido`: MySQL calcula el subtotal solo, así nunca queda mal:

```
subtotal DECIMAL(12,2) AS (cantidad * precio_unitario) STORED
```

Otros comandos DDL: `ALTER TABLE platos ADD COLUMN picante TINYINT(1);` (agregar una columna), `DROP TABLE` (borrar una tabla), `TRUNCATE` (vaciarla). El archivo `migracion_001_fotos.sql` es un ejemplo de `ALTER`: agregó las fotos a una base que ya existía.

DML: manipular los datos

DML (Data Manipulation Language) son los comandos que trabajan con **los datos**: `INSERT`, `SELECT`, `UPDATE`, `DELETE`.

```sql
-- INSERT: crear
INSERT INTO categorias (nombre, descripcion) VALUES ('Bebida', 'Jugos naturales');

-- SELECT: consultar
SELECT nombre, precio FROM platos WHERE disponible = 1 ORDER BY precio DESC;

-- UPDATE: modificar (¡siempre con WHERE!)
UPDATE platos SET precio = 30000 WHERE id = 5;

-- DELETE: eliminar (¡siempre con WHERE!)
DELETE FROM categorias WHERE id = 4;
```

> **OJO**
> Un `UPDATE` o `DELETE` **sin WHERE** afecta a **todas** las filas de la tabla.

Tipos de consultas que pide la lista de chequeo

Todas están en `database/consultas.sql`, listas para ejecutar.

Generales (sin filtros)

```sql
SELECT * FROM categorias;
SELECT * FROM platos ORDER BY nombre;
```

Específicas (con condiciones)

```sql
-- Búsqueda por texto: LIKE con % (cualquier cosa antes o después)
SELECT nombre FROM platos WHERE nombre LIKE '%leche%' OR descripcion LIKE '%leche%';
-- Rango de precios
SELECT nombre, precio FROM platos WHERE disponible = 1 AND precio BETWEEN 5000 AND 10000;
-- Pedidos en un estado
SELECT * FROM pedidos WHERE estado = 'pendiente';
```

Multitabla (JOIN)

Un **JOIN** une filas de dos o más tablas usando la relación (FK = PK).

```sql
-- INNER JOIN: solo las filas que tienen pareja en ambas tablas
SELECT p.nombre, p.precio, c.nombre AS categoria
FROM platos p
INNER JOIN categorias c ON c.id = p.categoria_id;

-- LEFT JOIN: TODAS las filas de la izquierda, aunque no tengan pareja
SELECT c.nombre, COUNT(p.id) AS total_platos
FROM categorias c
LEFT JOIN platos p ON p.categoria_id = c.id      -- incluye categorías sin platos (con 0)
GROUP BY c.id;

-- Detalle de un pedido uniendo 5 tablas
SELECT pe.id, u.nombre AS cliente, pl.nombre AS plato, c.nombre AS categoria,
       d.cantidad, d.precio_unitario, d.subtotal
FROM pedidos pe
INNER JOIN usuarios u        ON u.id = pe.usuario_id
INNER JOIN detalle_pedido d  ON d.pedido_id = pe.id
INNER JOIN platos pl         ON pl.id = d.plato_id
INNER JOIN categorias c      ON c.id = pl.categoria_id
WHERE pe.id = 1;
```

`p`, `c`, `pe`... son **alias**: nombres cortos para las tablas. `AS` renombra una columna en el resultado.

De resumen (funciones de agregación)

```sql
-- COUNT cuenta, SUM suma, AVG promedia, MIN y MAX
SELECT COUNT(*) AS pedidos, SUM(total) AS ventas, ROUND(AVG(total)) AS ticket_promedio
FROM pedidos WHERE estado <> 'cancelado';

-- GROUP BY agrupa: cuántos pedidos hay por cada estado
SELECT estado, COUNT(*) AS pedidos FROM pedidos GROUP BY estado;

-- Los 5 platos más vendidos
SELECT pl.nombre, SUM(d.cantidad) AS unidades
FROM detalle_pedido d
INNER JOIN platos pl ON pl.id = d.plato_id
GROUP BY pl.id, pl.nombre
ORDER BY unidades DESC
LIMIT 5;
```

Transacciones en SQL

```sql
START TRANSACTION;          -- empieza
INSERT INTO pedidos ...;
INSERT INTO detalle_pedido ...;
COMMIT;                     -- confirma todo   (o ROLLBACK; para deshacer todo)
```

La sección 5 de `consultas.sql` hace un INSERT, UPDATE y DELETE de demostración dentro de una transacción con `ROLLBACK`, para mostrarlos sin dañar los datos. La sección 6 trae sentencias que **deben fallar** para demostrar que las llaves foráneas protegen los datos (error 1451 y 1452).

Cómo se crea la base de datos

- `schema.sql`: DDL, crea la base y las 7 tablas.

- `seed.sql`: DML, inserta las 3 categorías y los 9 platos iniciales.
- `instalar.php`: ejecuta los dos archivos, crea el administrador con su contraseña cifrada y, con `--demo`, agrega clientes y pedidos ficticios. Usa `IF NOT EXISTS` e `INSERT IGNORE` para poder ejecutarse varias veces sin duplicar.
- `instalacion_hosting.sql`: versión para pegar en phpMyAdmin del hosting.

> **DILO ASÍ**
> "La base de datos es relacional, en MySQL, con 7 tablas. Las relaciones se garantizan con llaves foráneas: por ejemplo, `platos.categoria_id` apunta a `categorias.id` con ON DELETE RESTRICT, así que no se puede borrar una categoría que tenga platos. La relación muchos a muchos entre pedidos y platos se resuelve con la tabla intermedia `detalle_pedido`, que guarda el precio del momento de la compra. Con DDL creé la estructura y con DML manejo los datos; las consultas generales, específicas, multitabla y de resumen están en `consultas.sql`."

PARTE 9

Validaciones y seguridad

Doble validación

Cada formulario se valida **dos veces** con las mismas reglas:

1. **En el navegador** (`src/utils/validaciones.js`): para avisar al instante, sin esperar al servidor.
2. **En el servidor** (`api/lib/validaciones.php`): porque la validación del navegador **se puede saltar** (cualquiera puede enviar peticiones directo a la API). El servidor es la última palabra.

> **CLAVE**
> "Nunca hay que confiar en lo que llega del navegador". Por eso el servidor vuelve a validar y por eso el precio del pedido se toma de la base de datos.

Reglas del registro

Campo	Reglas
Nombre	Obligatorio, de 3 a 100 caracteres, solo letras (con tildes y ñ) y espacios
Correo	Obligatorio, formato válido (nombre@dominio.com), máximo 150, no repetido
Teléfono	Obligatorio, de 7 a 15 dígitos, puede empezar con +
Contraseña	Obligatoria, de 8 a 72 caracteres, debe combinar letras y números

`src/utils/validaciones.js (fragmento)`

```
if (!datos.correo) errores.correo = 'El correo es obligatorio.'
else if (!REGEX_CORREO.test(datos.correo)) {
  errores.correo = 'Escriba un correo válido, por ejemplo nombre@correo.com.'
}
if (datos.password.length < 8 || datos.password.length > 72) {
  errores.password = 'La contraseña debe tener entre 8 y 72 caracteres.'
} else if (!/[A-Za-z]/.test(datos.password) || !/\d/.test(datos.password)) {
  errores.password = 'La contraseña debe combinar letras y números.'
}
```

Los mensajes de error se muestran **debajo de cada campo** gracias al componente `Campo.jsx`.

Otras validaciones: precio del plato mayor que 0, categoría existente, cantidad de cada plato entre 1 y 50, foto JPG/PNG/WEBP de máximo 2 MB.

Login seguro

- **Contraseñas cifradas (hash):** nunca se guarda la contraseña real. `password_hash()` la convierte en un texto irreversible (bcrypt) y `password_verify()` la compara al iniciar sesión. Ni el administrador puede ver las contraseñas.
- **Límite de intentos (contra fuerza bruta):** tras 5 intentos fallidos con el mismo correo desde el mismo equipo, se bloquea 15 minutos (código 429). Se guardan en la tabla `intentos_login`.
- **Mensaje genérico:** "Correo o contraseña incorrectos", para no revelar si el correo existe.

- **Sesión en cookie segura:** `httpOnly` (JavaScript no la puede leer), `SameSite=Strict` (no se envía desde otros sitios), dura 8 horas y se regenera al iniciar sesión.

Permisos (autorización)

Se revisan en **dos lugares**: en React (`RutaProtegida`, para no mostrar pantallas) y, sobre todo, en PHP (`requiereSesion()`, `requiereAdmin()`), que es lo que realmente protege. Un cliente solo puede ver **sus propios** pedidos.

> **AUTENTICACIÓN VS. AUTORIZACIÓN**
> **Autenticación:** ¿quién eres? (login). **Autorización:** ¿qué puedes hacer? (rol cliente o admin).

Resumen de amenazas y defensas

Amenaza	Defensa en el proyecto
Inyección SQL	Consultas preparadas con PDO (`?`)
Robo de contraseñas	Hash con `password_hash`
Fuerza bruta en el login	Límite de intentos y bloqueo de 15 minutos
XSS (inyectar código en la página)	React escapa el texto que muestra; cookie `httpOnly`
Un cliente haciendo cosas de admin	`requiereAdmin()` en cada ruta protegida (403)
Subir archivos peligrosos	Se revisa el tipo real del archivo, el tamaño y las dimensiones; se guarda con un nombre generado
Claves en el código	Credenciales en `.env`, que no se sube a GitHub
Cambiar el precio desde el navegador	El servidor toma el precio de la base de datos

> **DILO ASÍ**
> "Valido en el navegador para dar respuesta inmediata y vuelvo a validar en el servidor porque es la única validación confiable. Las contraseñas se guardan con hash, el login tiene límite de intentos contra fuerza bruta, uso consultas preparadas contra la inyección SQL y cada ruta del administrador revisa el rol en el servidor."

PARTE 10

La matriz CRUD

CRUD son las cuatro operaciones básicas sobre los datos: **C**reate (crear), **R**ead (leer), **U**pdate (actualizar) y **D**elete (eliminar). La lista de chequeo pide al menos un módulo al 100 %. El proyecto tiene **dos módulos completos**: platos y categorías.

Módulo Platos (100 %)

Operación	Pantalla (React)	Endpoint	SQL
Crear	Admin → Platos → "Nuevo plato" (`PlatoFormulario.jsx`)	POST /api/platos	INSERT INTO platos
Read (consultar)	Menú, lista del admin con filtros, detalle (`AdminPlatos`, `PlatoDetalle`)	GET /api/platos, GET /api/platos/{id}	SELECT … INNER JOIN categorias
Update (editar)	"Editar" (`PlatoFormulario.jsx`, el mismo formulario)	PUT /api/platos/{id}	UPDATE platos SET …
Delete (eliminar)	Botón "Eliminar" con confirmación	DELETE /api/platos/{id}	DELETE FROM platos

Extra: subir y quitar la foto del plato (tabla `archivos`). Un plato que ya se vendió no se puede eliminar (se sugiere marcarlo "no disponible").

Módulo Categorías (100 %)

Operación	Pantalla	Endpoint	SQL
C	`AdminCategorias.jsx` (formulario)	POST /api/categorias	INSERT
R	Tabla con cantidad de platos	GET /api/categorias	SELECT … LEFT JOIN platos … GROUP BY
U	Botón "Editar"	PUT /api/categorias/{id}	UPDATE
D	Botón "Eliminar" (solo si no tiene platos)	DELETE /api/categorias/{id}	DELETE

Otros módulos

Módulo	C	R	U	D
Usuarios	Registro	Sesión actual	—	—
Pedidos	Confirmar orden	Mis pedidos / gestión admin	Cambiar estado / cancelar	— (se cancelan, no se borran, para conservar el historial)
Reportes	—	Resumen de ventas	—	—

DILO ASÍ

"El CRUD de platos está completo: el administrador crea un plato con su foto, lo consulta en la lista con filtros y en su detalle, lo edita con el mismo formulario y lo elimina con confirmación. Cada acción llama a un endpoint de la API (POST, GET, PUT, DELETE) que ejecuta el INSERT, SELECT, UPDATE o DELETE correspondiente. Lo mismo pasa con categorías."

PARTE 11

Recorrido completo: la vida de un pedido

Este recorrido une **todas las capas**. Si Valentina puede contarlo de memoria, entiende el proyecto.

#	Qué pasa	Capa	Dónde
1	La clienta abre la página. React pide `GET /api/platos` y muestra las tarjetas.	React + API + BD	`Menu.jsx` → `platos.php` → SELECT con JOIN
2	Toca "Agregar a la orden" en la bandeja paisa. Se ejecuta `agregarAlCarrito` y el estado `carrito` cambia.	JavaScript / React	`Tarjeta.jsx` → `App.jsx`
3	Aparece el carrito con el total calculado con `reduce`.	React	`Carrito.jsx`
4	Escribe una observación y toca "Confirmar pedido". Si no ha iniciado sesión, va al login.	React Router	`confirmarPedido` en `App.jsx`
5	Se envía `POST /api/pedidos` con `{ items: [{plato_id: 5, cantidad: 2}], observaciones }` en JSON y la cookie de sesión.	HTTP	`services/api.js`
6	Vite reenvía la petición a PHP. `index.php` encuentra la ruta `POST /pedidos`.	Enrutador	`vite.config.js`, `api/index.php`
7	`requiereSesion()` revisa que haya sesión; `validarPedido` revisa los datos.	Seguridad y validación	`seguridad.php`, `validaciones.php`
8	Empieza una **transacción**: lee los precios reales, calcula el total, hace INSERT en `pedidos` e INSERT en `detalle_pedido`. COMMIT.	PHP + MySQL	`pedidos.php`
9	Responde `201` con `{ "id": 12, "total": 56000, "estado": "pendiente" }`.	HTTP / JSON	`responder()`
10	React vacía el carrito y muestra "Pedido #12 confirmado. Ver pedido".	React	`Menu.jsx`
11	El administrador entra a Pedidos y cambia el estado a "En preparación": `PATCH /api/pedidos/12/estado` → `UPDATE pedidos SET estado = ?`.	Todas	`AdminPedidos.jsx`, `PedidoDetalle.jsx`
12	La clienta ve el nuevo estado en "Mis pedidos".	React + API	`MisPedidos.jsx`

Estados de un pedido

```
pendiente ──► en_preparacion ──► entregado
    │
    └──► cancelado   (el cliente solo puede cancelar mientras está "pendiente")
```

PRÁCTICA RECOMENDADA

Hacer este recorrido con F12 abierto en la pestaña **Network**: se ve la petición `pedidos`, el método POST, el cuerpo enviado, el código 201 y la respuesta JSON. Después mirar la tabla `pedidos` en phpMyAdmin y ver el registro nuevo.

PARTE 12

Mapa de archivos del proyecto

```
menu_restaurante/
├── index.html              Única página HTML; React se monta en <div id="root">
├── package.json            Librerías y comandos (npm run dev, api, build, db:setup…)
├── vite.config.js          Configuración de Vite y del proxy hacia PHP
├── .env.example            Plantilla de configuración (copiar como .env)
├── .htaccess               En Apache/hosting, envía todo a api/index.php
│
├── src/                    ── FRONTEND (React) ──
│   ├── main.jsx            Arranque: Router + Sesión + App
│   ├── App.jsx             Rutas de la aplicación y estado del carrito
│   ├── index.css           TODOS los estilos
│   ├── empresa.js          Misión, visión, valores, servicios, contacto
│   ├── Tarjeta.jsx         Tarjeta de un plato
│   ├── Filtro.jsx          Botones de categorías
│   ├── Carrito.jsx         La orden del cliente
│   ├── components/         Piezas reutilizables
│   │   ├── Navbar.jsx      Barra de navegación
│   │   ├── Campo.jsx       Campo de formulario con etiqueta y error
│   │   ├── Aviso.jsx       Mensajes de éxito/error
│   │   ├── EstadoPedido.jsx  Etiqueta de color del estado
│   │   └── RutaProtegida.jsx Guardia de rutas (sesión / admin)
│   ├── pages/              Pantallas
│   │   ├── Menu, Nosotros, Registro, Login, MisPedidos, PedidoDetalle
│   │   └── admin/          AdminPlatos, PlatoFormulario, PlatoDetalle,
│   │                       AdminCategorias, AdminPedidos, Reportes
│   ├── context/            Sesión compartida (sesion.js, SesionProvider.jsx)
│   ├── services/api.js     ÚNICO lugar donde se llama a la API
│   └── utils/              validaciones.js, formato.js (pesos y fechas)
│
├── api/                    ── BACKEND (PHP) ──
│   ├── index.php           Punto de entrada y enrutador
│   ├── lib/                config, db, respuesta, seguridad, validaciones, archivos
│   └── rutas/              auth, categorias, platos, pedidos, reportes
│
├── database/               ── BASE DE DATOS ──
│   ├── schema.sql          DDL: crea la base y las 7 tablas
│   ├── seed.sql            DML: categorías y platos iniciales
│   ├── consultas.sql       Consultas para la sustentación
│   ├── instalar.php        Instalador (npm run db:setup)
│   ├── migracion_001_fotos.sql   ALTER para agregar las fotos
│   └── instalacion_hosting.sql   Versión para phpMyAdmin del hosting
│
├── tests/api.test.js       8 pruebas automáticas de la API (npm run test:api)
└── docs/                   Documentación técnica, manual de usuario, capturas, QR
```

OJO

Archivos de la plantilla que no se usan: `src/App.css`, `src/assets/react.svg`, `src/assets/vite.svg`, `src/assets/hero.png`. El favicon (`public/favicon.svg`) es el logo de Vite, no el del restaurante.

Pruebas

`tests/api.test.js` tiene 8 pruebas automáticas que llaman a la API real y comprueban: consultas públicas, registro y login con validaciones, bloqueo por fuerza bruta, permisos, CRUD de platos y categorías, pedidos con transacción y subida de fotos. Se ejecutan con `npm run test:api` (con la API encendida) y todas pasan.

PARTE 13

Preguntas probables de los profesores

Practicar respondiéndolas **en voz alta**, sin leer.

Generales

1. ¿Qué tecnologías usó y por qué?

> React para la interfaz porque permite componentes reutilizables y una navegación rápida sin recargar; PHP para la API porque es el lenguaje de servidor que vimos y funciona en cualquier hosting; MySQL porque es una base de datos relacional gratuita y muy usada, ideal para datos con relaciones como pedidos y platos.

2. ¿Cuál es la arquitectura del sistema?

> Cliente-servidor en tres capas: frontend en React, backend como API REST en PHP y base de datos MySQL. Se comunican con HTTP y JSON.

3. ¿Qué diferencia hay entre frontend y backend?

> El frontend se ejecuta en el navegador y es lo que el usuario ve; el backend se ejecuta en el servidor, aplica las reglas y accede a la base de datos. El usuario nunca ve el código del backend.

HTML y CSS

4. ¿Dónde está el HTML?

> En `index.html` está la estructura base, y el resto está escrito en JSX dentro de los componentes de React, que se convierte en HTML real en el navegador. Se puede ver con F12.

5. ¿Qué etiquetas semánticas de HTML5 usó?

> header, nav, main, section, article y footer, además de form, label, table, ul/li, y campos con tipos email, password, number, date, search, file y checkbox.

6. ¿Cómo hizo que el sitio se vea bien en celular?

> Con la etiqueta meta viewport y media queries en CSS. Por debajo de 768px aparece un botón "Menú" que despliega la navegación, el carrito pasa debajo del menú y las tablas se convierten en tarjetas. También uso Flexbox con flex-wrap y Grid con auto-fit para que las columnas se acomoden solas.

7. ¿Qué diferencia hay entre Flexbox y Grid?

> Flexbox acomoda elementos en una sola dirección (fila o columna); Grid acomoda en dos dimensiones (filas y columnas a la vez).

JavaScript y React

8. ¿React es un framework?

> Técnicamente es una librería de JavaScript para construir interfaces. Se le llama framework porque, junto con React Router y Vite, cumple ese papel en el proyecto.

9. ¿Qué es un componente? ¿Qué son las props y el estado?

> Un componente es una función que devuelve lo que se va a mostrar. Las props son los datos que recibe del componente padre (no los cambia); el estado son datos internos que cambian con useState y, al cambiar, actualizan la pantalla.

10. ¿Para qué sirve useEffect?

> Para ejecutar código cuando el componente aparece o cuando cambian ciertos datos. Lo uso para pedir los platos a la API cada vez que cambia la categoría o la búsqueda.

11. ¿Cómo funciona el carrito?

> Es un estado en App.jsx que guarda un arreglo de platos con su cantidad. Con find reviso si el plato ya está; si está, con map le sumo 1; si no, lo agrego. Con filter lo quito y con reduce calculo el total. El carrito vive en la memoria del navegador, así que se borra si se recarga la página.

12. ¿Cómo protege las páginas del administrador?

> En React con el componente RutaProtegida, que redirige al login o muestra "No tiene permisos". Pero la protección real está en el servidor: cada ruta de administrador llama a requiereAdmin() y responde 403 si el usuario no es admin.

PHP y API

13. ¿Qué es una API REST?

> Un conjunto de direcciones (endpoints) que el servidor ofrece para trabajar con recursos como platos o pedidos, usando los métodos HTTP: GET para consultar, POST para crear, PUT/PATCH para editar y DELETE para eliminar. Responde en JSON.

14. ¿Cómo se conecta PHP con MySQL?

> Con PDO, en `api/lib/db.php`. Los datos de conexión se leen del archivo .env. Todas las consultas son preparadas: los valores van aparte en los signos de pregunta.

15. ¿Qué es la inyección SQL y cómo la evita?

> Es cuando un atacante escribe código SQL en un campo para que la base de datos lo ejecute. La evito con consultas preparadas: el valor nunca se mezcla con la orden SQL.

16. ¿Qué es una transacción y dónde la usa?

> Un grupo de operaciones que se ejecutan todas o ninguna. La uso al crear el pedido: se guarda el pedido y su detalle; si algo falla, se hace ROLLBACK y no queda nada a medias.

17. ¿Cómo guarda las contraseñas?

> Con password_hash de PHP, que genera un hash bcrypt irreversible. Al iniciar sesión, password_verify compara lo escrito con el hash. Nunca se guarda la contraseña real.

Base de datos

18. ¿Cuántas tablas tiene y cómo se relacionan?

> Siete: categorias, platos, archivos, usuarios, pedidos, detalle_pedido e intentos_login. Una categoría tiene muchos platos, un usuario muchos pedidos, y pedidos y platos tienen una relación muchos a muchos resuelta con detalle_pedido. Un plato puede tener una foto en archivos. intentos_login no se relaciona porque solo registra intentos de inicio de sesión.

19. ¿Qué diferencia hay entre llave primaria y foránea?

> La primaria identifica de forma única cada fila de una tabla. La foránea es una columna que apunta a la llave primaria de otra tabla y crea la relación.

20. ¿Qué diferencia hay entre DDL y DML? Dé ejemplos.

> DDL define la estructura: CREATE TABLE, ALTER TABLE, DROP. DML maneja los datos: INSERT, SELECT, UPDATE, DELETE. El DDL está en schema.sql y el DML en seed.sql, consultas.sql y en la API.

21. ¿Qué diferencia hay entre INNER JOIN y LEFT JOIN?

> INNER JOIN trae solo las filas que tienen pareja en las dos tablas. LEFT JOIN trae todas las de la tabla izquierda aunque no tengan pareja. Uso LEFT JOIN para contar los platos de cada categoría, así aparecen también las categorías con 0 platos.

22. ¿Qué pasa si intento borrar una categoría que tiene platos?

> No se puede: la llave foránea tiene ON DELETE RESTRICT. Además la API lo revisa antes y responde con un mensaje claro (409).

23. ¿Por qué detalle_pedido guarda el precio?

> Para conservar el precio histórico: si el precio del plato cambia después, el pedido sigue mostrando lo que el cliente pagó.

Validaciones y CRUD

24. ¿Qué validaciones tiene?

> Campos obligatorios, longitudes mínimas y máximas, formato de correo y teléfono con expresiones regulares, contraseña de 8 caracteres con letras y números, correo no repetido, precio mayor que cero, cantidades de 1 a 50 y tipo y tamaño de la foto. Se validan en el navegador y otra vez en el servidor.

25. Muéstreme el CRUD completo de un módulo.

> (Demostrar en vivo con Platos: crear con foto → ver en la lista y en el menú → editar el precio → eliminar.) Explicar que cada botón llama a POST, GET, PUT o DELETE en la API.

PARTE 14

Glosario

Término	Significado
API	Conjunto de "puertas" que un programa ofrece para que otro lo use.
Asíncrono	Operación que tarda y no bloquea el resto del programa (como pedir datos).
Backend	Parte que corre en el servidor.
Build	Convertir el código de desarrollo en archivos listos para publicar (`dist/`).
Componente	Pieza reutilizable de la interfaz en React.
Cookie	Dato pequeño que el navegador guarda y reenvía al servidor.
CRUD	Crear, leer, actualizar y eliminar.
DDL / DML	Comandos SQL de estructura / de datos.
Endpoint	Una dirección concreta de la API (método + ruta).
Framework	Estructura completa que organiza una aplicación y llama a tu código.
Frontend	Parte que corre en el navegador.
Hash	Transformación irreversible de un texto (para contraseñas).
Hook	Función especial de React que empieza por "use" (useState, useEffect).
HTTP	Protocolo de comunicación de la web.
JSON	Formato de texto para intercambiar datos.
JSX	Sintaxis para escribir HTML dentro de JavaScript en React.
Librería	Código reutilizable que tu programa llama cuando lo necesita.
Media query	Regla CSS que se aplica según el tamaño de la pantalla.
npm	Gestor de paquetes de Node.js.
PDO	Extensión de PHP para conectarse a bases de datos.
Props	Datos que un componente padre pasa a un hijo.
Proxy	Intermediario que reenvía peticiones (Vite → PHP).
Regex	Expresión regular: patrón para validar textos.
Responsive	Diseño que se adapta a cualquier tamaño de pantalla.
Sesión	Información que el servidor recuerda de un usuario conectado.
SPA	Aplicación de una sola página que cambia de pantalla sin recargar.
SQL	Lenguaje para consultar y manejar bases de datos.
Estado	Datos internos de un componente que, al cambiar, actualizan la pantalla.
Transacción	Grupo de operaciones SQL que se ejecutan todas o ninguna.

Autoevaluación final

Marca cada punto solo cuando puedas explicarlo **sin mirar**:

	Puedo explicar…
☐	Las tres capas del sistema y cómo se comunican.
☐	Qué es HTML, qué etiquetas semánticas y controles de formulario uso, y qué es JSX.
☐	Qué es un selector CSS, el modelo de caja, Flexbox, Grid y las media queries.
☐	Las funciones del carrito (find, map, filter, reduce) y qué es async/await.
☐	Si React es librería o framework, y qué son componentes, props, estado, useEffect, Context y rutas.
☐	Los métodos HTTP, los códigos 200/201/400/401/403/404/409 y qué es JSON.
☐	Cómo funciona el enrutador de PHP y el patrón permisos → validar → SQL → responder.
☐	Qué es PDO, qué son las consultas preparadas y la inyección SQL.
☐	Las 7 tablas, sus relaciones, PK, FK y las reglas RESTRICT/CASCADE/SET NULL.
☐	La diferencia entre DDL y DML con ejemplos, y escribir un SELECT con JOIN y GROUP BY.
☐	Las validaciones del registro y por qué se valida en el navegador y en el servidor.
☐	Cómo se protegen las contraseñas y el login.
☐	El CRUD completo de platos y de categorías (y demostrarlo en vivo).
☐	La vida de un pedido, paso a paso, del clic a la base de datos.

CONSEJO FINAL

Si un profesor pregunta algo que no sabes, no inventes: di qué parte del sistema se encarga de eso y muéstralo en el código o en la aplicación. Mostrar el sistema funcionando (y la pestaña Network de F12) vale más que cualquier definición memorizada.